

Qintern-2024 Project 01: Bias-preserving gate

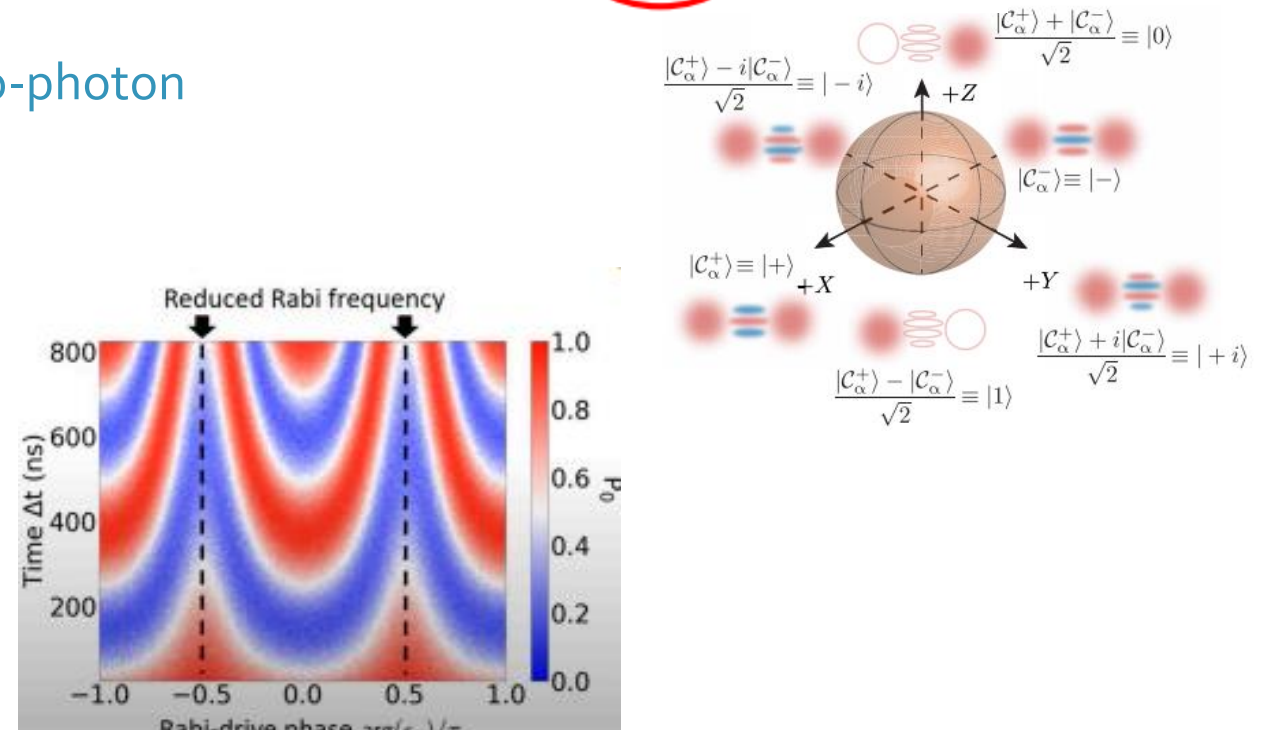
Slides prepared by: Sadia Humaira Salsabil

Kerr-Cat States

- To get a coherent state from a fock state: a single photon squeezing drive + detuning
- To get Cat states (two coherent states): two photon squeezing drive + Kerr non-linearity
- To apply transformations: phase of the two-photon drive should be changed
- For cat states, the bit flips are suppressed.
The dominant error is phase flip

$$\mathbf{H}_{\text{coh}}/\hbar = -\Delta \mathbf{a}^\dagger \mathbf{a} + \epsilon(\mathbf{a}^\dagger + \mathbf{a})$$

$$\mathbf{H}_{\text{KC}}/\hbar = -K \mathbf{a}^{\dagger 2} \mathbf{a}^2 + \epsilon_2(\mathbf{a}^{\dagger 2} \mp \mathbf{a}^2)$$



Conventional CX Gate

Let's say the noise channel is biased, and the dominant error is the Phase flip

When **phase flip** occurs at the target qubit, the phase flip propagates

- as a **phase** flip in the control qubit
- as a combination of both **phase** and **bit** flip in the target qubit

$$\hat{U}(T) = \left[\left(\frac{\hat{I}_1 + \hat{Z}_1}{2} \right) \otimes \hat{I}_2 + \left(\frac{\hat{I}_1 - \hat{Z}_1}{2} \right) \otimes \hat{X}_2 \right]$$

$$\begin{aligned} \hat{U}_e(T) &= \hat{U}(T - \tau) \hat{I}_1 \otimes \hat{Z}_2 \hat{U}(\tau) \\ &= \hat{I}_1 \otimes \hat{Z}_2 e^{iV(T-\tau)(\hat{I}_1 - \hat{Z}_1) \otimes \hat{X}_2} \hat{U}(T) \end{aligned}$$

Bias preserving CX gate

- In order to remove Bit flip error, the CX gate should not contain the X operator
- For Cat states, to go from $|\mathcal{C}_\alpha^\pm\rangle$ to $|\mathcal{C}_{\alpha e^{i\pi}}^\pm\rangle$, the phase phi of the two-photon drive should be changed from 0 to π

$$\begin{aligned}
 & c_0|0\rangle \otimes \{(d_0 + d_1)|\mathcal{C}_\alpha^+\rangle + (d_0 - d_1)|\mathcal{C}_\alpha^-\rangle\} + \\
 & c_1|1\rangle \otimes \{(d_0 + d_1)|\mathcal{C}_{\alpha e^{i\pi}}^+\rangle + (d_0 - d_1)|\mathcal{C}_{\alpha e^{i\pi}}^-\rangle\} \quad \longrightarrow \quad c_0|0\rangle \otimes (d_0|0\rangle + d_1|1\rangle) + c_1|1\rangle \otimes (d_0|1\rangle + d_1|0\rangle) \\
 & \quad \quad \quad = \hat{U}_{CX}|\psi(0)\rangle
 \end{aligned}$$

$$\hat{P}_C \hat{H}_{ZZ} \hat{P}_C = J_{12} \alpha^2 \hat{Z}_1 \hat{Z}_2 + J_{12} \alpha^2 e^{-2\alpha^2} \hat{Y}_1 \hat{Y}_2$$

```

N=2

g_noise=10
c_ops = [ g_noise * tensor(qeye(N),sigmaz())] # Dephasing
e_ops= [sigmax(), sigmaz()]

g=100
e=100

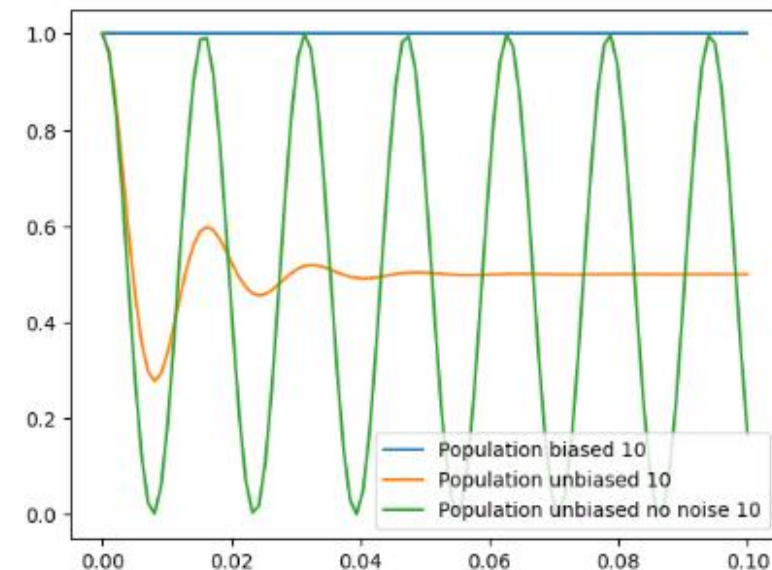
H_biased=g*tensor(sigmaz(),sigmaz())
H_biased_nonoise=g*tensor(sigmaz(),sigmaz())
H_unbiased=g*tensor(sigmaz(),sigmaz())+e*tensor(sigmay(),sigmay())
H_unbiased_nonoise=g*tensor(sigmaz(),sigmaz())+e*tensor(sigmay(),sigmay())

tlist=linspace(0,0.1,100)

initial_state= ket2dm(tensor(fock(N,1),fock(N,0)))

result_biased=mesolve(H_biased,initial_state, tlist, c_ops , [])
result_unbiased=mesolve(H_unbiased,initial_state, tlist, c_ops , [])
result_unbiased_nonoise=mesolve(H_unbiased_nonoise,initial_state, tlist, [], [])

```



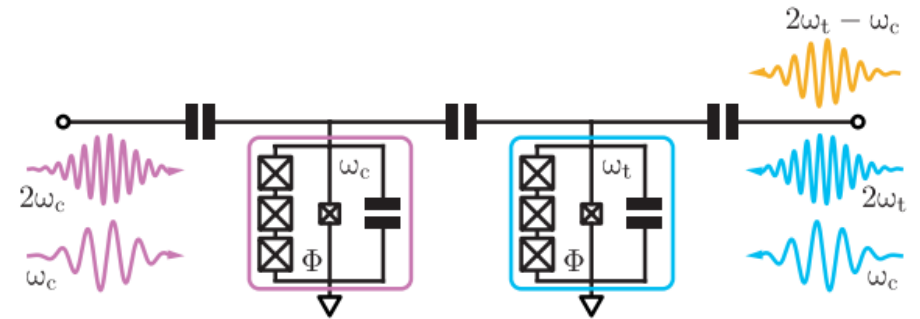
Hamiltonian of Bias preserving CX gate

$$CX = |\alpha\rangle\langle\alpha| \otimes I_2 + |-\alpha\rangle\langle-\alpha| \otimes Z_2$$

$$\begin{aligned} \hat{H}_{CX} = & -K(\hat{a}_c^{\dagger 2} - \beta^2)(\hat{a}_c^2 - \beta^2) - \\ & K \left[\hat{a}_t^{\dagger 2} - \alpha^2 e^{-2i\phi(t)} \left(\frac{\beta - \hat{a}_c^{\dagger}}{2\beta} \right) - \alpha^2 \left(\frac{\beta + \hat{a}_c^{\dagger}}{2\beta} \right) \right] \\ & \times \left[\hat{a}_t^2 - \alpha^2 e^{2i\phi(t)} \left(\frac{\beta - \hat{a}_c}{2\beta} \right) - \alpha^2 \left(\frac{\beta + \hat{a}_c}{2\beta} \right) \right] - \\ & \frac{\dot{\phi}(t)}{4\beta} \hat{a}_t^{\dagger} \hat{a}_t (2\beta - \hat{a}_c^{\dagger} - \hat{a}_c) \end{aligned}$$

Required Drives

$$\begin{aligned} \hat{H} = & -K\hat{a}_c^{\dagger 2}\hat{a}_c^2 - K\hat{a}_t^{\dagger 2}\hat{a}_t^2 + K\beta^2(\hat{a}_c^{\dagger 2} + \text{h.c.}) + K\alpha^2\cos(\phi(t))(e^{i\phi(t)}\hat{a}_t^{\dagger 2} + \text{h.c.}) - \\ & (K\alpha^2\sin(\phi(t))/\beta)(ie^{i\phi(t)}\hat{a}_t^{\dagger 2}\hat{a}_c + \text{h.c.}) + (K\alpha^4/2\beta)\sin(2\phi(t))(i\hat{a}_c^{\dagger} + \text{h.c.}) - \\ & (K\alpha^4\sin^2(\phi(t))/\beta^2)\hat{a}_c^{\dagger}\hat{a}_c - \phi(t)\hat{a}_t^{\dagger}\hat{a}_t/2 + (\phi(t)/4\beta)\hat{a}_t^{\dagger}\hat{a}_t(\hat{a}_c^{\dagger} + \text{h.c.}) \end{aligned}$$



Squeezing drives

Pump at control
 $\epsilon_p = K\alpha^2 \cos(2\omega_c t)$

Pump at target
 $\epsilon_p = K\beta^2 \cos(2\omega_t t + \Phi(t))$

Multi-photon exchange interaction

$$(K\alpha^2\sin(\phi(t))/\beta)(ie^{i\phi(t)}\hat{a}_t^{\dagger 2}\hat{a}_c + \text{h.c.})$$

Pump at target
 $\epsilon_p = \frac{K\alpha^2\sin(\Phi(t))}{\beta} \cos((2\omega_t - \omega_c)t + \Phi(t) + \frac{\pi}{2})$

Conditional Displacement

$$(\phi(t)/4\beta)\hat{a}_t^{\dagger}\hat{a}_t(\hat{a}_c^{\dagger} + \text{h.c.})$$

Pump at target
 $\epsilon_p = \frac{\dot{\Phi}(t)}{4\beta} \cos(\omega_c t)$

Single-photon drive at control

$$\epsilon_p = \frac{K\alpha^4\sin(\Phi(t))}{2\beta} \cos((2\omega_c)t + \frac{\pi}{2})$$

Detuning at control and target

$$\begin{aligned} \Delta_c &= -\frac{K\alpha^4}{\beta^2} \sin^2(\Phi(t)) \\ \Delta_t &= \frac{\dot{\Phi}(t)}{2} \end{aligned}$$

Simulation example

Parameters taken from SCQubits

Interaction strength

$$-\epsilon_p E_j \frac{g_{1,c}}{w_1 - w_c} \sqrt{(w_1/2E_l)} \frac{g_{2,c}}{w_2 - w_c} \sqrt{(w_2/2E_l)}$$

$$\text{where, } g_{i,c} = \frac{e^2}{C_{i,c}} \sqrt{\sqrt{\frac{E_{l,i} E_{l,c}}{16 E_{c,1} E_{c,c}}}}$$

$$w_i = \sqrt{8 E_{c,i} E_{l,i}}$$

$$H_{driver} = g_3(a^{\dagger 2}b + a^2b^{\dagger})$$

$$H_{driver} = -2E_j \cos(\phi + \phi_{\Delta}) \cos(\phi_{\Sigma})$$

```
g3 = metrics["g_3"]
a=qt.tensor(qt.destroy(8), qt.qeye(5), qt.qeye(8))
c=qt.tensor(qt.qeye(8), qt.destroy(5), qt.qeye(8))
b=qt.tensor(qt.qeye(8), qt.qeye(5), qt.destroy(8))

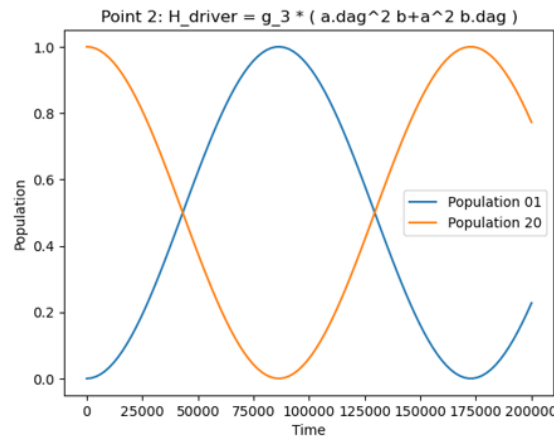
H_static=wa*a.dag()*a+ wb*b.dag()*b
H_driver_g3= g3 * (a.dag() * a.dag() * b + a*a*b.dag())
```

```
H_driver_g3_ti = - 2 * Ej_c * (phi+phi_del_ext).cosm()

def H_driver_g3_lab(tlist3):
    return np.cos(eps_p*np.cos((wp*tlist3))+phi_sum_ext)*H_driver_g3_ti

def H_driver_g3_rot(tlist3):
    U=((1j* wa*a.dag()*a*tlist3)+(1j*wb*b.dag()*b*tlist3)).expm()
    rot1=H_driver_g3_lab(tlist3)*U.dag()
    rot2=U*rot1
    return rot2
```

H_driver only



Challenges Faced

- The Hamiltonian using drives in rotating frame took too long to simulate

References

Shruti Puri *et al.*, Bias-preserving gates with stabilized cat qubits. *Sci. Adv.* **6**, eaay5901 (2020). DOI: [10.1126/sciadv.aay5901](https://doi.org/10.1126/sciadv.aay5901)

Grimm, A., Frattini, N.E., Puri, S. *et al.* Stabilization and operation of a Kerr-cat qubit. *Nature* **584**, 205–209 (2020). <https://doi.org/10.1038/s41586-020-2587-z>

Thank you